PID-RS TECHNICAL REPORT SERIES

Exact-Count Executable Assurance for the Complete Averaged Two-Source Categorical Shared-Exclusions Decomposition

Conditional certification, adversarial qualification, and an explicit refinement boundary

pid-rs project analysis

28 July 2026

Abstract

This document specifies and audits the standalone exact-count reference certifier implemented in `audit/tools/certified-sxpid`. The tool reconstructs the empirical two-source categorical shared-exclusions partial information decomposition of Makkeh, Gutknecht, and Wibral from exact positive integer counts. It emits all 24 averaged cumulative and Möbius-atom coordinates: informative, misinformative, and signed-net components for four lattice nodes and four atoms. Event masses, weights, logarithm arguments, and Möbius transforms are exact integers or rationals. A deliberately isolated Rug/MPFR wrapper evaluates each canonical log-linear expression with explicit downward and upward rounding and exports exact dyadic endpoints. The mathematical enclosure argument is given step by step. Resource preflights, exact identities, adaptive interval intersection, and a strict certificate schema remain the magnitude-enclosure route. Report v2 additionally clears the empirical denominator and, when a tighter preflight admits it, compares one exact rational product with one to certify zero or strict sign separately from the interval. The retained total-count-eight counterexample demonstrates why empty canonical terms were not a complete zero criterion. A corpus of 494 exhaustive small empirical laws, 13,338 numerical comparisons, metamorphic tests, and 34 fail-closed static-policy mutations provide additional but distinct evidence.

A second standard-library Python implementation now treats the producer certificate as untrusted. It independently reconstructs the events, exact rational expressions, fixed lattice, and three direct mutual-information identities. It then proves containment with integer fixed-point outward bounds for a rational logarithm series. Qualification reconstructs 11,856 coordinates, checks 1,482 direct-MI identities and 5,928 independent direct row-scan event-expression identities over the complete 494-table bounded domain, exactly compares all 11,856 coordinate products, proves 72 containments from three live certificates, checks 975 exact-`Fraction` logarithm enclosures, kills 23 semantic mutations, one fixed-point source mutation, and one event-extraction source mutation, rejects four cross-artifact binding adversaries and six structural adversaries, and passes two transport/invocation controls plus two loaded-execution cache/code controls. A typed mutation sweep changes and restores each of the 51 declared semantic/configuration globals and requires the integrity guard to reject every mutation. On the affected CPython 3.11 route, a separate control kills a source mutant that removes the normalization call. Independent verification schema v3 first normalizes CPython's nonsemantic string-intern cache state before serializing live module-owned code and a deterministic typed encoding of that exact 51-name inventory. This repairs a retained CPython 3.11 false rejection and closes four hostile-review omissions without discarding the post-import code-mutation guard. The route treats producer-supplied expressions, lattice values, signs, and endpoints as untrusted inputs and removes their correctness—together with Rug, MPFR, GMP, rustc, and the producer executable—from the independent containment trusted computing base. Producer endpoints necessarily remain the interval tested by subset

inclusion. Python integer and `Fraction` semantics, JSON and SHA-256, the verifier source, the logarithm proof, and the locally bound producer sources remain trusted. An additional 111-mutation claim-custody suite attacks schema and source bindings, structured Markdown, historical revisions, machine evidence, workflow and Just containers, static policy, PDF leaf gates, assurance sources, and raw-byte line-ending drift. It is change-control evidence, not a proof that the reviewed inventory is complete.

The result is intentionally narrower than "formally verified pid-rs." Neither route refines the `pid-core` binary64 implementation, establishes statistical or population validity, covers pointwise or higher-source SxPID, or validates continuous PID or any downstream decision. Concrete negative counterexamples are retained to prevent those invalid inferences.

Contents

1 Reader orientation: what is being decomposed

This section supplies the mathematical context needed for the assurance argument. No familiarity with partial information decomposition (PID), shared exclusions, Rust, or interval arithmetic is assumed.

1.1 From mutual information to four PID atoms

Let S_1 and S_2 be two categorical sources and let T be a categorical target. Ordinary mutual information $I(S_1, S_2; T)$ measures how statistically informative the pair of sources is about the target. It does not say whether that information is available from either source alone, only from one particular source, or only from the pair.

A two-source PID names four atoms, whose signed-net values need not be nonnegative:

- redundancy R, available from either source;
- unique information U_1, available only from S_1;
- unique information U_2, available only from S_2; and
- synergy S, available only from the joint source (S_1, S_2).

They must reconstruct the three ordinary mutual informations:

$$I(S_1; T) = U_1 + R, \tag{1}$$
$$I(S_2; T) = U_2 + R, \tag{2}$$
$$I(S_1, S_2; T) = U_1 + U_2 + S + R. \tag{3}$$

These three equations contain four unknowns. A redundancy definition is therefore required before the four atoms are determined. This document uses the categorical shared-exclusions definition of Makkeh, Gutknecht, and Wibral; it does not propose a new PID.

1.2 Pointwise shared-exclusion cumulatives

Fix an observed realization $z = (s_1, s_2, t)$. A source event says which other rows agree with the source part of z. For example, $E_1(z)$ contains rows whose first-source value is s_1. The shared-exclusions redundancy event is the disjunction

$$E_\vee(z) = E_1(z) \cup E_2(z) : \tag{4}$$

a row belongs when it matches at least one source. The joint-source event is instead the intersection $E_{12}(z) = E_1(z) \cap E_2(z)$. Confusing these two events changes the functional; Counterexample 8.1 exhibits the difference.

For any one of the four source events $A_\alpha(z)$, define the pointwise informative, misinformative, and signed-net cumulatives

$$c_\alpha^+(z) = -\ln P(A_\alpha(z)) , \tag{5}$$

$$c_\alpha^-(z) = -\ln P(A_\alpha(z) \mid T = t) = \ln \frac{P(T = t)}{P(A_\alpha(z) \cap \{T = t\})}, \tag{6}$$

$$c_\alpha^{\mathrm{sx}}(z) = c_\alpha^+(z) - c_\alpha^-(z) = \ln \frac{P(A_\alpha(z) \cap \{T = t\})}{P(A_\alpha(z))P(T = t)}. \tag{7}$$

The superscript sx labels the signed shared-exclusions net component. The natural logarithm is used throughout, so every information value is measured in nats. "Pointwise" means that the quantity is keyed by one realization z; "averaged" means its expectation

$$C_\alpha^u = \sum_z P(z) c_\alpha^u(z), \qquad u \in \{+, -, \mathrm{sx}\}. \tag{8}$$

The certifier evaluates the empirical version obtained by replacing P with the exact empirical law determined by integer counts.

1.3 Why there are four cumulative nodes

For a nonempty source collection $a \subseteq \{1, 2\}$, define

$$E_a(z) = \bigcap_{i \in a} E_i(z).$$

An antichain is a nonempty set of nonempty source collections in which no member contains another. The complete two-source antichain set is

$$\alpha_R = \{\{1\}, \{2\}\}, \qquad \alpha_1 = \{\{1\}\}, \qquad \alpha_2 = \{\{2\}\}, \qquad \alpha_{12} = \{\{1, 2\}\}.$$

Its source event is the union

$$A_\alpha(z) = \bigcup_{a \in \alpha} E_a(z).$$

The redundancy order is

$$\alpha \preceq \beta \quad \Longleftrightarrow \quad \forall b \in \beta \; \exists a \in \alpha : \; a \subseteq b.$$

Thus the covers are

$$\alpha_R \prec \alpha_1 \prec \alpha_{12}, \qquad \alpha_R \prec \alpha_2 \prec \alpha_{12},$$

while α_1 and α_2 are incomparable. In the executable notation the four cumulative nodes are $1, 2, 12, \vee$, corresponding respectively to $\alpha_1, \alpha_2, \alpha_{12}, \alpha_R$. The symbol $\vee$ denotes an event union but is the *bottom* of this redundancy order; it must not be read as the lattice supremum.

For each component $u \in \{+, -, \mathrm{sx}\}$, zeta reconstruction means

$$C_\alpha^u = \sum_{\beta \preceq \alpha} \Pi_\beta^u.$$

In the ordinary atom names, the four equations are

$$C_1^u = U_1^u + R^u, \tag{9}$$
$$C_2^u = U_2^u + R^u, \tag{10}$$
$$C_{12}^u = U_1^u + U_2^u + S^u + R^u, \tag{11}$$
$$C_\vee^u = R^u. \tag{12}$$

Solving these four linear equations gives

$$R^u = C_\vee^u, \qquad U_1^u = C_1^u - C_\vee^u, \qquad U_2^u = C_2^u - C_\vee^u, \tag{13}$$
$$S^u = C_{12}^u - C_1^u - C_2^u + C_\vee^u. \tag{14}$$

Section 4 records the same calculation as mutually inverse integer matrices. The calculation is performed separately for informative, misinformative, and net components; exact arithmetic also checks that net equals informative minus misinformative.

1.4 A complete two-row example

Consider two equally weighted rows,

$$(S_1, S_2, T) = (0,0,0), \qquad (1,1,1). \tag{15}$$

For either keyed row, all four source events and the target event select that row alone. Hence $P(A_\alpha) = P(T = t) = P(A_\alpha \cap \{T = t\}) = 1/2$. Equations (5)–(7) give

$$c_\alpha^+ = \ln 2, \qquad c_\alpha^- = 0, \qquad c_\alpha^{\mathrm{sx}} = \ln 2 \tag{16}$$

for every node. Averaging changes nothing. Equation (12) therefore gives

$$R^{\mathrm{sx}} = \ln 2, \qquad U_1^{\mathrm{sx}} = U_2^{\mathrm{sx}} = S^{\mathrm{sx}} = 0. \tag{17}$$

Both sources are identical copies of the target, so the result is entirely redundant, as expected. This example explains the mathematical object; it is not evidence that an implementation is correct.

1.5 A complete negative-atom example

Net shared-exclusions atoms are signed and are never clamped. Let S_1, S_2 be independent uniform bits and $T = S_1$ xor S_2. The empirical law has four rows,

$$(0,0,0), \quad (0,1,1), \quad (1,0,1), \quad (1,1,0),$$

each with probability $1/4$. For every supported row, the four cumulative coordinates are

Node	$P(A_\alpha)$	$P(A_\alpha \cap \{T = t\})$	c_α^+	c_α^-	c_α^{sx}
$\vee$	$3/4$	$1/4$	$\ln(4/3)$	$\ln 2$	$\ln(2/3)$
1	$1/2$	$1/4$	$\ln 2$	$\ln 2$	0
2	$1/2$	$1/4$	$\ln 2$	$\ln 2$	0
12	$1/4$	$1/4$	$\ln 4$	$\ln 2$	$\ln 2$

Applying the four inversion formulas separately to each component gives

Atom	Informative	Misinformative	Net
R	$\ln(4/3)$	$\ln 2$	$\ln(2/3)$
U_1	$\ln(3/2)$	0	$\ln(3/2)$
U_2	$\ln(3/2)$	0	$\ln(3/2)$
S	$\ln(4/3)$	0	$\ln(4/3)$

All four rows have these same pointwise values, so the values are also the averaged atoms. They reconstruct

$$\ln \frac{2}{3} + 2 \ln \frac{3}{2} + \ln \frac{4}{3} = \ln 2 = I((S_1, S_2); T),$$

while $R^{\mathrm{sx}} + U_i^{\mathrm{sx}} = 0 = I(S_i; T)$. The informative and misinformative atoms are separately nonnegative for this functional, but their signed difference need not be. Calling all net atoms positive "contributions" or clipping $R^{\mathrm{sx}} = \ln(2/3)$ would change the paper-defined quantity and break these identities.

For completeness, that componentwise nonnegativity has a short event-algebra proof. Apply the following calculation first to the unconditional source-event law (the informative component) and then to the conditional law given $T = t$ (the misinformative component). Write

$$a = P(E_1 \cap E_2), \qquad b = P(E_1 \setminus E_2), \qquad c = P(E_2 \setminus E_1),$$

where the keyed observation gives $a > 0$. The bottom redundancy atom is nonnegative because it is minus the logarithm of an event probability. Möbius inversion gives

$$U_1 = \ln \frac{a+b+c}{a+b} \geq 0, \qquad U_2 = \ln \frac{a+b+c}{a+c} \geq 0, \tag{18}$$

$$S = \ln \frac{(a+b)(a+c)}{a(a+b+c)} = \ln \left(1 + \frac{bc}{a(a+b+c)}\right) \geq 0. \tag{19}$$

The conditional calculation is legitimate because every keyed conditional denominator is positive. Averaging these pointwise inequalities preserves nonnegativity. Subtracting the two component atoms, however, need not preserve it, as the XOR redundancy demonstrates.

1.6 What a certificate means

Each exact averaged coordinate is a finite rational linear combination of logarithms of positive rationals. The producer encloses it in a dyadic interval, and the independent verifier reconstructs the expression from counts and proves that its own enclosure is contained in the producer interval. Such an interval is a *numerical enclosure of an empirical functional*. It is not a sampling confidence interval, a population statement, a causal conclusion, or a downstream safety score. The rest of this paper proves and qualifies only this narrow executable-containment claim.

2 Status, provenance, and permitted claim

Record status. Historical claim-packet revision 1 records the interval-only producer and independent verifier at commit `b8b9a48b88cb28d812d8cbd70b8f999a3bac5a8e`. Revision 2 retains report schema v2 and resource policy v2 while adding the separately bounded exact-product decision described here. Revision 3 retains those producer semantics but changes the independent verification schema to v3: its loaded-execution digest normalizes nonsemantic CPython string-intern cache state before serialization. Revisions 1 and 2 remain historical evidence and are not silently rewritten. Publication binding for revision 3 is the full containing Git commit; this paper is neither external custody nor an executable attestation.

2.1 What is paper-defined and what is project-defined

The categorical shared-exclusions functional and its informative and misinformative event construction are paper-defined by Makkeh, Gutknecht, and Wibral [1]. This document claims no new PID measure and no scientific priority for that functional.

The following are project-defined assurance work:

- the exact-count input and certificate schemas;

- the reviewed two-source event extractor and fixed matrix encoding;
- the exact rational log-linear intermediate representation;
- the adaptive directed-rounding wrapper and exact dyadic output;
- the bounded denominator-cleared exact rational-product sign decision;
- resource and failure contracts;
- digest and build-context evidence;
- bounded exhaustive, metamorphic, and mutation qualification;
- the independent exact-integer and rational-log containment verifier; and
- the claim boundary recorded in every successful certificate.

The machine-readable authority is `method-catalog.json`. The source contract is the standalone tool document at `audit/tools/certified-sxpid/README.md`. The tool is an unpublished Cargo workspace rather than a `pid-core` backend.

2.2 Human-readable restatement of the permitted claim

The current byte-exact assurance boundary is pinned in `claims/SX-CERTIFIED-AVERAGED-PID2-001/bindings-v3.md` and in the independent verifier source. The following prose is a human-readable restatement; it is not the byte-binding itself.

The producer alone supports the following conditional statement:

> For the recorded two-source empirical count table, definition revision, four-node lattice, precision policy, source wrapper, locked dependency versions, and explicitly non-exhaustive build context, every emitted dyadic interval encloses the exact-real value of the log-linear expression encoded by the certifier, conditional on the unverified Rug, MPFR, GMP, compiler, native ABI, effective build configuration, and input-meaning trust boundary. When a coordinate's separate exact-product record has status `compared`, that record additionally certifies exact zero or strict sign by exact rational comparison after integer denominator clearing. The dyadic endpoints remain the enclosure authority, and the interval-local decision is not rewritten.

An accepted independent-verifier report supports a different, stronger numerical statement:

> For the same canonical count table and locally bound producer source manifest, the verifier independently reconstructs the 24 exact log-linear coordinates from counts and proves that its own integer/Fraction logarithm enclosure is contained in every producer interval. Conditional on the reviewed verifier source and logarithm proof, the Python loader and arbitrary-precision semantics, JSON and SHA-256, and process integrity, the 24 producer intervals enclose those independently reconstructed exact-real coordinates. The verifier also independently reconstructs every admitted exact product, repeats its resource preflight, and checks the separate zero or strict-sign decision without treating that decision as an interval endpoint claim.

The second route does not trust Rug, MPFR, GMP, rustc, the producer executable, or the correctness of any producer-supplied expression, lattice, endpoint, or sign for containment. It parses those fields as untrusted inputs; the endpoints necessarily define the candidate interval in the subset test. It still does not prove that `pid-core` produced the same expression or the same finite-precision value.

2.3 Excluded claims

No result in this document establishes:

- correctness of `pid-core` binary64 output;
- pointwise SxPID, $I_{\min}$, fitted-quantizer equivalence, or three- and four-source SxPID;
- continuous KSG, continuous shared exclusions, or continuous PID;
- population support, sampling assumptions, consistency, calibration, or confidence coverage;
- authenticity, provenance, or scientific meaning of an input table;
- correctness of Rug, MPFR, GMP, rustc, the linker, the native compiler, or the ABI;
- executable or native-archive identity; or
- downstream scientific, monitoring, safety, or authorization validity.

3 Exact empirical specification

3.1 Canonical positive count table

Let the observed complete categorical states be

$$z = (s_1, s_2, t) \in \mathcal{Z}_+, \tag{20}$$

where $\mathcal{Z}_+$ is the finite set of states with positive empirical count. Each source state and target state may itself be a fixed-width vector of categorical tokens. Let

$$c_z \in \mathbb{N}_{>0}, \qquad N = \sum_{z \in \mathcal{Z}_+} c_z, \qquad \widehat{P}(z) = \frac{c_z}{N}. \tag{21}$$

Rows are strictly lexicographically ordered, unique, and width-consistent. Counts use canonical positive decimal strings. The parser rejects duplicate JSON keys, unknown fields, zero or noncanonical counts, unsorted or duplicate rows, inconsistent widths, unsupported schema values, and resource-policy violations.

The accepted structural domain is exact: there are between 1 and 4096 rows. The source-one, source-two, and target vectors each have a width between 1 and 32 tokens, fixed across all rows; the three widths need not equal one another. Every token has between 1 and 128 ASCII bytes and matches

`[A-Za-z0-9._:+-]+`. (22)

Every count matches `[1-9][0-9]*` and contains at most 1024 decimal digits. These syntax limits precede the separate aggregate count-bit, serialized-size, and term-count resource limits.

This representation lists positive empirical support only. It does not claim that the observed support equals an unknown population support.

3.2 Realization-keyed source events

Fix one supported realization $z = (s_1, s_2, t)$. Define

$$E_1(z) = \{(s_1', s_2', t') : s_1' = s_1\}, \tag{23}$$
$$E_2(z) = \{(s_1', s_2', t') : s_2' = s_2\}, \tag{24}$$
$$E_{12}(z) = E_1(z) \cap E_2(z), \tag{25}$$
$$E_\vee(z) = E_1(z) \cup E_2(z), \tag{26}$$
$$T(z) = \{(s_1', s_2', t') : t' = t\}. \tag{27}$$

Equality is equality of the complete token vector. The cumulative-node order is

$$(1,\ 2,\ 12,\ \vee), \tag{28}$$

implemented by source masks

$$([01],\ [10],\ [11],\ [01, 10]). \tag{29}$$

The last event is the source disjunction used by the shared-exclusions redundancy node. It is not the joint-source intersection.

For a node α, let $A_\alpha(z)$ be its source event and define exact integer masses

$$U_{\alpha,z} = \sum_{z' \in A_\alpha(z)} c_{z'}, \qquad V_{\alpha,z} = \sum_{z' \in A_\alpha(z) \cap T(z)} c_{z'}, \qquad T_z = \sum_{z' \in T(z)} c_{z'}. \tag{30}$$

The independent verifier reconstructs the redundancy masses by inclusion–exclusion. For the unrestricted source event,

$$U_{\vee,z} = U_{1,z} + U_{2,z} - U_{12,z}. \tag{31}$$

For the target-restricted event, strict uniqueness of complete states couples the intersection mass to the keyed row:

$$E_1(z) \cap E_2(z) \cap T(z) = \{z\} \quad \text{within the listed support}, \qquad V_{\vee,z} = V_{1,z} + V_{2,z} - c_z. \tag{32}$$

The equality uses the schema requirement that each complete (s_1, s_2, t) state occurs exactly once. A schema permitting repeated complete states would have to subtract their aggregated intersection mass rather than reuse c_z; that change would require a new proof and retained mutation.

Proposition 3.1 (Positive denominator chain)**.** *For every supported z and every implemented node α,*

$$0 < c_z \le V_{\alpha,z} \le U_{\alpha,z} \le N, \qquad V_{\alpha,z} \le T_z \le N. \tag{33}$$

Proof. The keyed realization belongs to every one of $E_1, E_2, E_{12}, E_\vee$ and to $T(z)$, so its positive count contributes to $V_{\alpha,z}$. Event inclusion gives the remaining inequalities. The implementation rechecks the corresponding exact integer inequalities for every realization–node pair and rejects on failure. □

3.3 Averaged informative, misinformative, and net cumulatives

The exact averaged cumulative quantities are

$$C_\alpha^+ = \sum_{z \in \mathcal{Z}_+} \frac{c_z}{N} \ln\left(\frac{N}{U_{\alpha,z}}\right), \tag{34}$$

$$C_\alpha^- = \sum_{z \in \mathcal{Z}_+} \frac{c_z}{N} \ln\left(\frac{T_z}{V_{\alpha,z}}\right), \tag{35}$$

$$C_\alpha^{\mathrm{sx}} = \sum_{z \in \mathcal{Z}_+} \frac{c_z}{N} \ln\left(\frac{N V_{\alpha,z}}{U_{\alpha,z} T_z}\right). \tag{36}$$

All coefficients and logarithm arguments are exact positive rationals. The extractor checks the local identity

$$\frac{N V_{\alpha,z}}{U_{\alpha,z} T_z} = \frac{N/U_{\alpha,z}}{T_z/V_{\alpha,z}} \tag{37}$$

before inserting any term. Therefore, over exact reals,

$$C_\alpha^{\mathrm{sx}} = C_\alpha^+ - C_\alpha^-. \tag{38}$$

For $\alpha \in \{1, 2, 12\}$, the net cumulative is separately reconstructed as ordinary empirical mutual information between the indicated source collection and the target. Equality is checked at the exact symbolic-expression level.

4 The fixed two-source lattice and exact intermediate form

4.1 Möbius and zeta matrices

For the antichains defined in the reader orientation, let

$$Z_{\alpha\beta} = \mathbf{1}\{\beta \preceq \alpha\}.$$

In cumulative row order $(\alpha_1, \alpha_2, \alpha_{12}, \alpha_R)$ and atom column order (U_1, U_2, S, R), this indicator definition gives the displayed Z below; its inverse is the displayed M. The matrices are therefore not arbitrary reconstruction coefficients.

For any component $u \in \{+, -, \mathrm{sx}\}$, collect cumulatives and atoms as

$$C^u = \begin{bmatrix} C_1^u & C_2^u & C_{12}^u & C_\vee^u \end{bmatrix}^\mathsf{T}, \qquad \Pi^u = \begin{bmatrix} U_1^u & U_2^u & S^u & R^u \end{bmatrix}^\mathsf{T}. \tag{39}$$

The certifier pins

$$\Pi^u = MC^u, \qquad M = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ -1 & -1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{40}$$

and

$$C^u = Z\Pi^u, \qquad Z = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{41}$$

Exact integer multiplication verifies $ZM = I_4$ at runtime. Reconstruction through Z is an arithmetic self-consistency check implied by the pinned inverse matrices; it is not independent semantic evidence for the node labels, atom labels, or event transcription. Subject to those semantic premises, every informative, misinformative, and net atom is reconstructed through Z and compared with its cumulative expression.

The defining paper applies Möbius inversion pointwise and then averages. The executable first averages the cumulatives and then applies M. These routes agree because M is a fixed finite linear map:

$$MC^u = M \sum_z \widehat{P}(z) c^u(z) = \sum_z \widehat{P}(z)\, M c^u(z).$$

This identity closes the averaged-atom bridge; it does not by itself validate the event transcription or the numerical logarithms.

4.2 Canonical log-linear expression

Each coordinate is represented as

$$F = \sum_{j=1}^{m} a_j \ln(q_j), \qquad a_j \in \mathbb{Q} \setminus \{0\}, \quad q_j \in \mathbb{Q}_{>0} \setminus \{1\}. \tag{42}$$

A sorted exact map is keyed by q_j. Equal arguments have their rational coefficients added. Zero coefficients and $\ln(1)$ are removed. This gives deterministic serialization and exact cancellation for identical arguments.

Proposition 4.1 (Historical v1 empty-map witness)**.** *If the canonical expression map is empty, then $F = 0$ exactly.*

Proof. Every retained term has been removed, so the represented finite sum is the empty sum. □

This was the only exact-zero witness in report schema v1. It is sound but incomplete. A retained empirical SxPID2 counterexample uses canonical binary-state counts

$$(0, 0, 1, 1, 1, 4, 1, 0), \qquad n = 8. \tag{43}$$

Its net unique-one atom has the nonempty expression

$$-\frac{1}{8}\ln\frac{8}{15} + \frac{1}{8}\ln\frac{4}{5} + \frac{1}{8}\ln\frac{8}{9} + \frac{1}{8}\ln\frac{4}{3} - \frac{1}{8}\ln\frac{16}{9}, \tag{44}$$

but denominator clearing gives the exact product

$$\left(\frac{8}{15}\right)^{-1}\frac{4}{5}\frac{8}{9}\frac{4}{3}\left(\frac{16}{9}\right)^{-1} = 1. \tag{45}$$

The live 256-bit directed interval is $[-3\,2^{-258}, 5\,2^{-259}]$, so its interval-local decision correctly remains `unresolved_sign`.

Report schema v2 adds a separate bounded exact-product record. For $F = \sum_j a_j \ln q_j$, empirical coefficients satisfy $e_j = na_j \in \mathbb{Z}$, and therefore

$$F = \frac{1}{n}\ln R, \qquad R = \prod_j q_j^{e_j}. \tag{46}$$

Since $n > 0$ and $R > 0$, exact zero, positivity, and negativity are respectively equivalent to $R = 1$, $R > 1$, and $R < 1$. This is exact rational comparison, not prime factorization. When its preflight admits the product, v2 records that decision separately; it never changes what the dyadic endpoints themselves establish.

4.3 All 24 coordinates

Expression order is fixed and complete:

1. 12 cumulatives: four nodes for each of informative, misinformative, and net; and
2. 12 atoms: four atoms for each of informative, misinformative, and net.

The tool rejects if extraction, evaluation, or certificate assembly does not contain exactly 24 coordinates.

5 Directed numerical enclosure

5.1 Arithmetic contract

Rug 1.30.0 supplies exact `Integer` and `Rational` values and an MPFR-backed `Float`. The transitive native binding is locked at `gmp-mpfr-sys` 1.7.1. The authoritative wrapper admits only explicit directed forms:

- exact-rational to MPFR conversion rounded downward or upward;
- natural logarithm rounded downward or upward;
- exact-rational multiplication rounded downward or upward; and
- accumulation rounded downward or upward.

MPFR defines rounding toward negative and positive infinity [2]; Rug exposes these operations and their rounding-order results [3]. The wrapper rejects forbidden rounding-order results, nonfinite endpoints, nonpositive converted logarithm arguments, and invalid interval order.

This is a contract with the arithmetic stack, not a proof of the arithmetic stack.

5.2 One-term enclosure

Let $q > 0$ and $a \neq 0$ be exact rationals. Directed conversion gives

$$q_\downarrow \leq q \leq q_\uparrow. \tag{47}$$

The wrapper rejects if either finite converted endpoint is nonpositive. Since ln is increasing,

$$\mathrm{RD}(\ln q_\downarrow) \leq \ln q \leq \mathrm{RU}(\ln q_\uparrow). \tag{48}$$

Write these logarithm bounds as ℓ and u.

If $a > 0$, multiplication gives

$$\mathrm{RD}(a\ell) \leq a \ln q \leq \mathrm{RU}(au). \tag{49}$$

If $a < 0$, endpoint order reverses:

$$\mathrm{RD}(au) \leq a \ln q \leq \mathrm{RU}(a\ell). \tag{50}$$

Counterexample 5.1 (Why negative coefficients must swap endpoints)**.** Suppose only that $x \in [1, 2]$ and let $a = -1$. The correct product interval is $[-2, -1]$. Reusing positive-coefficient endpoint order produces the invalid ordered pair $[-1, -2]$. Möbius inversion creates negative coefficients, so this is a central soundness condition rather than a numerical refinement.

5.3 Finite-sum enclosure theorem

Theorem 5.2 (Conditional outward enclosure)**.** *Let F be the exact expression in Equation (42). Assume every Rug/MPFR conversion, logarithm, multiplication, and addition obeys its stated directed-rounding contract. If the wrapper returns finite endpoints L and U without rejection, then*

$$L \leq F \leq U. \tag{51}$$

Proof. The preceding one-term construction encloses each exact $a_j \ln(q_j)$, with the endpoint swap when $a_j < 0$. Initialize the lower and upper sums at exactly zero. Inductively, if $L_k \leq \sum_{j=1}^{k} a_j \ln(q_j) \leq U_k$, downward-rounded addition of the next lower term cannot exceed the exact new sum, while upward-rounded addition of the next upper term cannot fall below it. The invariant therefore holds after every term and yields the claim at $k = m$. □

Corollary 5.3 (Conditional 24-coordinate certificate)**.** *Assume the exact event extractor, canonical expression construction, fixed matrices, and Rug/MPFR arithmetic stack behave as encoded. Every successful certificate interval encloses its corresponding tool-encoded cumulative or atom coordinate.*

Remark 5.4. Corollary 5.3 does not prove the antecedents. In particular, it is not a deductive Rust-to-SxPID refinement theorem.

5.4 Exact dyadic endpoints

Every finite MPFR endpoint is a dyadic rational. The wrapper obtains an exact integer significand and binary exponent and emits

$$d = m\,2^e, \qquad m \in \mathbb{Z}, \quad e \in \mathbb{Z}. \tag{52}$$

For nonzero d, factors of two are removed until m is odd. Zero is uniquely encoded with $m = 0, e = 0$. No JSON floating-point value is authoritative.

5.5 Adaptive precision and intersection

The default working precisions are

$$128,\ 256,\ 512,\ 1024,\ 2048,\ 4096 \text{ bits}. \tag{53}$$

At each precision, all 24 expressions are rebuilt from exact rationals and independently enclosed. Let $I_{j,p}$ be the interval for coordinate j at precision p. The accepted aggregate is the exact dyadic intersection

$$J_{j,k} = \bigcap_{r=1}^{k} I_{j,p_r}. \tag{54}$$

Proposition 5.5 (Intersection preserves enclosure)**.** *If $x_j \in I_{j,p_r}$ for every $r \le k$, then $x_j \in J_{j,k}$.*

Proof. Membership in every set is exactly membership in their intersection. □

An empty intersection is therefore an internal soundness failure under the stated arithmetic contract. It is never repaired by widening or selecting one interval.

Success requires every exact width to satisfy

$$\operatorname{width}(J_{j,k}) \le 2^{-160}. \tag{55}$$

Certificate assembly recomputes this rational predicate and rejects if it is false. Exhausting 4096 bits or six iterations returns a precision-limit rejection rather than a partial certificate.

5.6 Sign decisions

For an interval $[L, U]$:

- $L > 0$ gives `certified_positive`;
- $U < 0$ gives `certified_negative`;
- an empty canonical expression gives `certified_exact_zero`; and
- every other case gives `unresolved_sign`.

Counterexample 5.6 (Zero is not a strict sign)**.** The interval $[0, \varepsilon]$ does not prove strict positivity, and $[-\varepsilon, 0]$ does not prove strict negativity. Replacing $>$ by $\ge$ or $<$ by $\le$ would certify a sign that the exact value zero contradicts. Both mutations are explicit fail-closed tests.

6 Certificate, failure, and resource contracts

6.1 Success envelope

A successful compact JSON envelope records:

- raw-input and normalized semantic-input SHA-256 digests;
- exact terms and an expression digest for each coordinate;
- exact dyadic intervals, precision, width result, and sign decision;
- a separate bounded exact-product status, exact sign/zero decision when compared, product-one witness, and resource-preflight trace;
- lattice and precision-policy values and digests;
- exact extraction and numerical cross-check counts;

- a runtime source-manifest digest and standalone lockfile digest;
- locked crate versions, manifest-requested Rug features, and explicitly absent native and executable attestations;
- sanitized, non-exhaustive Cargo profile metadata; and
- permitted and excluded claims.

The accepted schemas are not left implicit. The six exact version identifiers are

Object	Required identifier
Input schema	`pid-rs/categorical-sxpid2-count-table/v1`
Definition revision	`makkeh-gutknecht-wibral-2021-empirical-sxpid2-v1`
Resource policy	`sxpid2-certification-default-v2`
Producer report schema	`pid-rs/certified-sxpid-report/v2`
Exact-expression schema	`pid-rs/exact-log-linear/v1`
Independent-verification schema	`pid-rs/certified-sxpid-independent-verification/v3`

The units field is exactly `nats`. The input object has exactly the keys

$$(\texttt{schema}, \texttt{definition\_revision}, \texttt{units}, \texttt{resource\_policy\_id}, \texttt{rows}),$$

and each row has exactly

$$(\texttt{source\_states}, \texttt{target\_state}, \texttt{count}).$$

The outer certificate envelope has exactly (`payload_sha256`, `payload`). Its payload has exactly

$$\begin{aligned}(&\texttt{schema}, \texttt{status}, \texttt{route},\\ &\texttt{definition\_revision}, \texttt{units}, \texttt{input},\\ &\texttt{exact\_expression}, \texttt{lattice}, \texttt{extraction\_checks},\\ &\texttt{precision\_policy}, \texttt{arithmetic}, \texttt{tool\_binding},\\ &\texttt{coordinates}, \texttt{cross\_checks}, \texttt{claim\_boundary}).\end{aligned}$$

Every coordinate has distinct `interval` and `exact_product` objects. The former is derived only from its dyadic endpoints. The latter reports `compared` plus an exact decision, or one of the two explicit preflight-unavailable statuses. A product-one witness may therefore refine an interval that straddles zero without rewriting the interval-local field. Unknown and duplicate object keys are rejected. JSON floating-point numbers and nonstandard numeric constants are prohibited. Integer-valued decimal strings must use the field-specific canonical grammar: no sign or leading zero is admitted where a positive count is required, and normalized dyadic integer fields use their separately specified signed canonical form.

The payload digest uses recursively lexicographic JSON object keys and no JSON floating-point numbers. The source-manifest digest length-prefixes every path and file payload under a domain tag. A static source-inventory gate prevents ordinary Rust modules or build-script helpers from silently escaping that manifest.

The independent verifier separately checks the local arithmetic source contract rather than treating rehashed manifest bytes as sufficient. It rejects Cargo `[patch]` and `[replace]` source-substitution tables, requires the standalone `[workspace]` table to remain empty, and pins both the registry source and registry checksum for the locked `rug` and `gmp-mpfr-sys` packages. Ambient Cargo configuration, downloaded registry custody, native archives, and executable bytes remain outside this local binding.

6.2 Failure semantics

The command emits a rejection envelope and a nonzero status for invalid input, internal invariant failure, resource rejection, or precision exhaustion. It emits no successful certificate after a rejected check. Consumers must require:

1. process exit status zero;
2. complete JSON through end of file;
3. the success schema and status;
4. a recomputed payload digest; and
5. acceptance of the embedded claim boundary.

The current command streams JSON to standard output. An external write failure can leave a truncated prefix before exit status 1. Such a prefix is not a valid complete certificate, but atomic transport is not established. This is a retained operational hardening item.

The independent verifier also treats output delivery as part of success. A retained POSIX closed-standard-output control requires a failed write or flush to return status 1 without a traceback or second interpreter-shutdown error. This is a fail-closed transport control; it does not make either output stream atomic.

6.3 Producer resource policy

Bound	Value	Rejection purpose
Input bytes	4 MiB	Bound parsing and raw-input retention
Positive rows	4096	Bound event scans and exact extraction
Source or target token width	32	Bound vector-state comparisons
Bytes per token	128	Bound string storage and comparison
Decimal digits per count	1024	Bound integer parsing
Total-count magnitude	8192 bits	Bound rational numerators and denominators
Terms per coordinate	4096	Bound one exact expression
All retained exact terms	8192	Bound the complete 24-coordinate representation
Cumulative extraction terms	$\lfloor 8192/5 \rfloor = 1638$	Reject before Möbius expansion
Estimated exact-term JSON	8 MiB	Reject before term materialization
Canonical payload	10 MiB	Bound final certificate serialization
Exact-product terms per expression	256	Bound one multiplicative comparison
Exact-product absolute exponent	16,384	Reject before rational powering

Bound	Value	Rejection purpose
Exact-product projected bits per expression	262,144	Reject before rational powering
Aggregate exact-product projected bits	1,048,576	Bound all admitted comparisons
Working precision	128–4096 bits, six iterations	Bound MPFR work
Required interval width	2^{-160}	Versioned acceptance threshold

The factor five in the cumulative term cap is structural. One cumulative term remains in the cumulative representation and can enter at most four atom expressions through one column of the pinned Möbius matrix. Hence

$$1638(1+4) = 8190 \leq 8192. \tag{56}$$

The incremental cumulative cap is checked while rows are extracted, before potentially large exact-term strings are serialized.

The exact-product projection is computed before any rational power:

$$B = \sum_j |e_j|(\mathrm{bits}(\mathrm{num}\, q_j) + \mathrm{bits}(\mathrm{den}\, q_j)), \qquad e_j = na_j. \tag{57}$$

A per-expression failure records `not_compared_per_expression_preflight_limit`; an aggregate failure records `not_compared_total_preflight_limit`. The interval certificate remains available, but no exact-product sign follows. The route uses exact rational powers only after admission and never invokes unrestricted integer factorization.

The 4096-row limit is a structural maximum, not a promise that every table of that size is accepted. A generic table can reach the cumulative-term ceiling much earlier: the retained 410-row growing-support witness transiently reconstructs 1640 cumulative terms and is rejected against the 1638-term ceiling. Checking only final expressions after cancellation would miss that resource amplification.

These bounds constrain memory-shaped objects and iteration counts, not wall-clock time. The producer currently performs eight complete event-mass scans for every accepted row, so extraction is $O(r^2)$ in the number r of support rows, in addition to vector-comparison and arbitrary-precision-integer costs. No latency, throughput, or denial-of-service-resistance claim follows from the policy.

6.4 Independent-verifier resource policy

The independently implemented Python verifier has a separate, smaller numerical schedule and additional parser and source-custody limits. These are verifier acceptance conditions, not complexity theorems:

Verifier bound	Value	Rejection purpose
Accepted input bytes	4 MiB	Bound input parsing
Producer-certificate bytes	11 MiB	Bound untrusted certificate parsing
Bytes per state token	128	Match the canonical input contract

Verifier bound	Value	Rejection purpose
Total-count magnitude	8192 bits	Bound exact rational arithmetic
Exact-product terms per expression	256	Match producer product preflight
Exact-product absolute exponent	16,384	Match producer product preflight
Exact-product projected bits per expression	262,144	Match producer product preflight
Aggregate exact-product projected bits	1,048,576	Match producer product preflight
Absolute dyadic exponent	65,536	Bound shifts and endpoint decoding
Fixed-point precision	at most 2048 bits	Bound integer-series evaluation
Fixed-point schedule	256, 384, 512, 768, 1024, 1536, 2048 bits	Bound retries
Report integer-string digits	4096	Bound parsed certificate integers
JSON numeric-integer-token digits	24	Bound numeric-token parsing
Canonical payload bytes	10 MiB	Match the producer payload contract
One source-manifest member	4 MiB	Bound source hashing
Complete source manifest	32 MiB	Bound manifest reconstruction
Verifier source bytes	2 MiB	Bound self-binding and source loading

The verifier also enforces the producer's row, token-width, count-digit, term-count, and cumulative-extraction limits during independent reconstruction. Reaching the end of the fixed precision schedule without proving subset containment is a rejection, not an approximate success.

7 Qualification evidence

7.1 Exact and metamorphic checks

The test suite includes:

- $ZM = I_4$ and exact zeta reconstruction for all three components as arithmetic self-consistency, not independent event-semantic evidence;
- exact local net-ratio and three self-redundancy mutual-information identities;
- analytic XOR coordinate checks;
- exact logarithm reciprocal and power identities;
- the nonempty five-term exact-product-one empirical counterexample and a huge-exponent preflight abstention before powering;

- overlap of selected low- and high-precision enclosures;
- tiny-positive sign resolution and nonresolution at different precisions;
- exact subtraction with crossed endpoints;
- source-one, source-two, and target vector-width metamorphisms;
- common scaling by a 1000-digit positive count;
- duplicate-key, schema, canonicalization, and precision-limit failures;
- process-level input, usage, success, and oversized-standard-input contracts; and
- early rejection of an adversarial dense, large-integer table.

7.2 Exhaustive small empirical laws

For binary (S_1, S_2, T) there are eight complete states. The independent standard-library generator enumerates every nonempty weak count composition with total sample size at most four:

$$\sum_{n=1}^{4} \binom{n+8-1}{8-1} = 8 + 36 + 120 + 330 = 494. \tag{58}$$

For every table it evaluates the published event construction directly using 80-digit Python `Decimal`, performs the fixed Möbius transform, and checks its own identities.

The certifier qualification performs

$$494 \times 24 = 11{,}856 \tag{59}$$

interval-versus-10^{-70}-tolerance overlap comparisons and

$$494 \times 3 = 1{,}482 \tag{60}$$

additional comparisons against separately evaluated direct mutual-information formulas. The total is 13,338 comparisons. Fixture and generator bytes are bound by reviewed literal SHA-256 values in the Rust test; the sidecar is not trusted by itself.

Remark 7.1 (Evidence boundary)**.** Finite `Decimal` values are not outward-rounded proof intervals. Tolerance overlap is bounded diverse-path agreement, not proof of enclosure, universal correctness, external authorship, or population validity.

Counterexample 7.2 (Overlap is weaker than containment)**.** Suppose the true value is 3, a faulty claimed interval is $[0, 2]$, and an approximate-reference interval is $[1.9, 3.1]$. The two intervals overlap even though the claimed interval excludes the truth. The exhaustive corpus is valuable bug-finding evidence because its tolerances are tiny and its cases complete within the bound, but overlap alone cannot carry the soundness theorem.

7.3 Independent-verifier bounded qualification

Separate from the Decimal-overlap lane, the Python qualification harness enumerates the same 494 bounded tables and reconstructs

$$494 \times 24 = 11{,}856$$

coordinates and

$$494 \times 3 = 1{,}482$$

direct-MI identities. It also checks

$$494 \times 12 = 5{,}928$$

cumulative event-expression identities by an independent direct row scan. That scan evaluates the four source predicates and target restriction row by row; it does not reuse the verifier's inclusion–exclusion shortcut in Equation (32).

This third event path kills one retained event-extraction source mutation: replacing

$$V_{1,z} + V_{2,z} - c_z \quad \text{by} \quad \max(V_{1,z}, V_{2,z}).$$

The mutant still passes the internal nesting checks, the three direct-MI identities, and the symmetric XOR pin. The 5,928 direct row-scan identities reject it, so those other identities are not presented as complete event-semantic evidence.

The same qualification independently clears every empirical denominator and compares all 11,856 coordinate products. It classifies 5,886 exact zeros, 5,762 positives, and 208 negatives, and checks every separate report decision against that result without changing an interval-local decision. A second exhaustive boundary covers all 12,869 nonzero binary tables with total at most eight (308,856 coordinates). It finds the first nonempty product-one expressions at total eight: exactly 16 five-support, five-term net unique atoms. The minimized witness is passed through the live certifier; its interval remains unresolved while its separate product record certifies exact zero. A resealed false product sign is rejected.

The recorded qualification tallies are:

Evidence class	Count
Reconstructed coordinates over 494 exhaustive tables	11,856
Direct-MI identities over those tables	1,482
Independent direct row-scan event-expression identities	5,928
Exact coordinate-product comparisons over 494 tables	11,856
Exact product decisions: zero / positive / negative	5,886 / 5,762 / 208
Boundary tables / coordinates through total eight	12,869 / 308,856
Nonempty product-one cases at the minimal bounded total	16
Live-certificate containments over three tables	72
Exact-`Fraction` logarithm-enclosure containments	975
Killed certificate/input semantic mutations	23
Killed fixed-point source mutations	1
Killed event-extraction source mutations	1
Rejected cross-artifact binding adversaries	4
Rejected structural adversaries	6
Passed transport/invocation controls	2
Passed loaded-execution cache/code controls	2
Rejected declared semantic/configuration-global mutations	51
Passed historical-receipt replay-projection controls	51
Outer receipt scalar-leaf mutations: changed / declared invariant	274 / 2
Certificate scalar-leaf mutations: changed / declared invariant	956 / 4
Killed CPython 3.11 cache-normalization source mutants	1 on the affected route

The 72 live containments are the 24 coordinates of singleton, XOR, and asymmetric sparse certificates. The 975 logarithm cases are 325 reduced rational arguments at each of 64, 128, and 256 bits; an exact-`Fraction` partial sum and rational tail enclosure must be contained in the fixed-point result.

The 23 semantic mutations comprise 22 self-consistently resealed certificate mutations and one changed-input reuse. One deliberately minimal containment mutation selects a certified-positive coordinate and changes only its upper endpoint to its own reported downward-rounded

lower endpoint before resealing the payload. Normalization, width, and the positive sign remain plausible; the independent subset-containment check rejects the collapsed interval. Two additional mutations exercise the strict interval/product boundary directly: a strict positive product paired with an interval whose upper endpoint is zero, and a strict negative product paired with an interval whose lower endpoint is zero. Both must fail before either endpoint equality can be mistaken for consistency. The separate exact-product self-test also runs two plan-before-powering sentinels. It replaces the auxiliary rational-power primitive with a fail-on-call function and confirms that one local preflight rejection and one aggregate rejection make zero power calls. These are ordering controls and are not included in the exact-product adversary total.

The boundary command also carries 51 distinct historical-receipt projection controls. Ordinary execution is read-only and emits a complete fresh receipt to standard output. The outer evidence comparison excludes only executable and full-certificate digests. Its replacement certificate projection first validates the complete payload digest and exact outer, tool, and build-context inventories, then removes only runtime source-manifest, Rust-version, build-host, and build-target leaves. Every other payload field remains equality-bound. Literal-inventory, excluded-leaf, retained-leaf, scientific-field, missing/extra, malformed-shape, and wrong-digest controls make that policy failure-sensitive. The retained same-host old/new certificates differed only in runtime source-manifest and outer payload digest and agreed on the narrowed projection; no second platform was executed. This is a declared variable-field projection boundary, not portable executable identity or cross-platform validation. Writing the historical receipt requires the explicit `-update-evidence` custody transition. Beyond the 51 targeted controls, an exhaustive recorded-schema pass mutates all 1,236 scalar leaves individually. The outer receipt partitions into 274 changed and exactly two invariant leaves; the certificate payload partitions into 956 changed and exactly four invariant leaves. Thus no other recorded scalar leaf escapes the appropriate projection. This does not exhaust structural rewrites, future schemas, parser faults, or cryptographic failure.

The four cross-artifact adversaries are post-import verifier-source drift, a rehashed Rug-version change, a rehashed Cargo `[patch]` substitution, and a rehashed locked-Rug-checksum change. The six structural adversaries are duplicate JSON keys, a lone Unicode surrogate, the 410-row transient-term witness, insufficient Python integer-text capacity for a producer-valid count, an invalid POSIX filename byte through the CLI, and process-start integer-text insufficiency. The two transport/invocation controls exercise a symlinked verifier script and closed standard output. The qualification harness compiles and executes the exact verifier source bytes without consulting a bytecode cache. Each negative control must fail through its intended guard; an unrelated rejection is not counted.

The two loaded-execution cache/code controls separate a harmless interpreter cache transition from an actual code mutation. The first loads two isolated copies with byte-identical probe code. The cold copy receives a dynamically constructed, initially non-interned string; its digest normalization intentionally primes that state. A distinct equal string is then explicitly interned, which must return the cold string object, before it is attached to the warm copy and the warm digest is taken. The two normalized digests must match. Isolation prevents the first digest from warming the same module object before the second initial-state condition is established. The second control replaces a live function's `__code__` object after import and requires the existing integrity guard to reject it through the intended error.

The digest also binds an exact, reviewed inventory of 51 uppercase module-owned semantic/configuration globals by name and value. A deterministic typed encoding distinguishes null, integers, text, bytes, lists, tuples, string-keyed mappings, and regular-expression pattern/flag pairs. Qualification mutates and restores every one of the 51 globals, including all four exact-product admission limits omitted by the first revision-3 candidate, and requires each mutation to trigger the intended integrity failure. Automatic discovery plus equality against the explicit 51-name qualification inventory makes a new uppercase configuration global fail closed until the

inventory and evidence are reviewed together.

The normalization recursively primes strings reachable through code metadata and constants before using the default runtime marshal format. It does not canonicalize bytecode across Python versions, attest the interpreter, cover mutate–use–restore races, bind trusted imports or unenumerated function state, or prove that every future behavior-affecting value will be declared as an uppercase configuration global. On CPython 3.11, a separate source-mutation control removes the normalization call, establishes a fresh qualification baseline for the otherwise unchanged mutant, performs the named intern-cache transition, and requires the loaded-execution guard to reject it. Other Python versions report that version-conditioned lane as unexercised rather than converting it into evidence.

7.4 Claim-packet and evidence-custody hostile controls

The revision-3 claim checker is intentionally a second assurance layer over the numerical verifier. Semantic checks bind schemas, exact evidence counts, source rows, claim boundaries, gate commands, and the one reviewed certified-SxPID method. Complete raw-byte digests separately bind:

- the retained revision-1 and revision-2 packet, all revision-3 authorities, and the incident record;
- the complete GitHub Actions and Just execution containers, not only the visible command substrings;
- the operator documents, formal-PDF dispatcher and both invoked leaves, static-policy checker and self-test, and the standalone deny policy; and
- the reviewed assurance Markdown, TeX sources, and committed PDFs.

Canonical whole-object projections bind the certified method-catalog object and all six machine-readable exact-product evidence records without freezing unrelated catalog methods. Strict JSON parsing rejects duplicate keys and nonstandard non-finite constants.

The machine inventory in `scripts/check-certified-sxpid2-claim-self-test.py` contains 111 named negative controls and runs under normal and optimized Python. Its attacks include invalid or equivalent Markdown fences, headings, links, entities, raw HTML, comments and code spans; contradictory historical, catalog, and evidence claims that preserve required substrings; workflow job disablement, global Just shell replacement, early exits in dispatchers and child gates; semantic source-row swaps; and an LF-to-CRLF raw-byte mutation. Each attack must reach a relevant fail-closed diagnostic. The full-byte bindings make an intentional change require explicit re-adjudication; they do not prove the correctness of SHA-256, Git, the filesystem, Python, TeX, PDF renderers, or the completeness of the mutation corpus, and they are not external custody.

7.5 Build modes and static policy

The complete standalone suite passes in:

- debug mode;
- optimized release mode;
- the repository's Rust 1.89 minimum supported toolchain; and
- the current stable toolchain with clippy, rustdoc, formatting, and dependency/license policy.

The static gate isolates authoritative Rug `Float` and `Round` use to one module, pins the exact directed-operation inventory, fixes event masks and matrices, validates the standalone source

inventory, checks the default locked dependency-feature graph, and rejects a direct native-system-feature injection surface. It also forbids unsafe blocks, functions, implementations, and traits in the standalone Rust source.

Its self-test kills 34 representative mutations:

Mutation family	Count
Arithmetic, rounding, sign, unsafe surface, and wrapper isolation	12
Source closure, compile-time inclusion, and build routing	10
Dependency-feature and evidence-claim boundary	2
Vector and count-event semantics, plus lattice algebra	6
Resource and report acceptance	4
Total	34

Mutation adequacy is not program verification. The suite proves only that these registered faults make the policy fail.

8 Retained negative counterexamples

Negative examples are permanent evidence. Removing them because the implementation currently passes would erase the reason for several guards.

8.1 Source disjunction is not a joint-source event

Counterexample 8.1 (XOR realization)**.** Take the four equally weighted XOR states

$$(0,0,0),\ (0,1,1),\ (1,0,1),\ (1,1,0). \tag{61}$$

For the realization $(0,0,0)$,

$$|E_1 \cup E_2| = 3, \qquad |E_1 \cap E_2| = 1. \tag{62}$$

The informative local values are therefore $\ln(4/3)$ and $\ln(4)$, respectively. Replacing the redundancy union mask $[01, 10]$ by the joint mask $[11]$ changes the defined functional.

8.2 Vector equality cannot be shortened

Counterexample 8.2 (Shared prefix)**.** The target states `["class","0"]` and `["class","1"]` are distinct. Comparing only their first token merges target events and changes T_z and $V_{\alpha,z}$. The same issue applies to vector-valued source states. Qualification therefore includes asymmetric vector-width metamorphisms and policy mutations that introduce shortcuts.

8.3 A digest sidecar cannot authenticate itself

Counterexample 8.3 (Coupled fixture mutation)**.** If an adversary changes a fixture and recomputes its adjacent checksum sidecar, checking only the pair accepts the coupled change. Literal expected digests for both fixture and generator bytes are therefore compiled into the reviewed test. This detects drift from those reviewed bytes; it does not establish authorship or external transparency.

8.4 Cargo features are additive

Cargo feature unification means another direct dependency edge can enable a transitive native feature. A prior direct `gmp-mpfr-sys` dependency would have created a command-line surface for enabling `use-system-libs`. The standalone manifest now depends only on Rug, and the qualification gate requires the locked default graph to resolve only the intended MPFR feature and requires direct feature injection to fail.

This closes one concrete injection surface. It does not bind effective compiler flags, wrappers, linker selection, native cache contents, archive bytes, or the executed binary. The runtime report states that absence rather than turning manifest intent into an attestation.

8.5 A complete emitted prefix is not a certificate

Transport success is part of acceptance. A truncated output prefix, a JSON document received with nonzero process status, or a payload whose digest does not recompute is rejected regardless of how many plausible coordinate fields it contains.

9 Assurance case and trusted computing base

9.1 Claim-to-evidence map

Claim fragment	Supporting evidence	What remains trusted
Count events have positive denominators	Proposition 3.1; exact runtime constraints; schema tests	Event definition and extractor implementation
Atoms reconstruct cumulatives	Fixed M, Z; exact $ZM = I$ and zeta arithmetic self-consistency	Semantic identification of node and atom order and events
One encoded expression is enclosed	Theorem 5.2; isolated directed wrapper; rounding-order checks	Rug, MPFR, GMP, compiler, ABI
One admitted exact-product record has the stated zero or strict sign	Integer denominator clearing; exact rational powers; numerator–denominator comparison; independent `Fraction` reconstruction	Producer or verifier implementation, according to the accepted route; preflight-unavailable coordinates abstain
The producer interval contains the independently reconstructed exact coordinate	Independent count-to-event extraction, fixed lattice, exact rational expressions, integer/Fraction log enclosure, and interval containment	Python runtime and loader, verifier source, logarithm proof, JSON, SHA-256, and local producer-source custody
All 24 intervals meet width policy	Adaptive engine; exact intersections; assembly recheck	Correct coordination of expressions and intervals
Small binary laws agree with a diverse route	494 complete bounded tables and 13,338 comparisons	Decimal implementation and the limitation of the bound

Claim fragment	Supporting evidence	What remains trusted
Independent bounded reconstruction is internally challenged	11,856 coordinates, 1,482 direct-MI identities, 5,928 direct row-scan identities, 72 live containments, and 975 exact-`Fraction` log containments	Verifier implementation and all domains outside the recorded grids and tables
Named producer fault classes fail closed	34 static-policy mutation self-tests	All unenumerated producer faults
Named independent-verifier faults fail closed	Intended-path semantic, structural, source, binding, transport, and loaded-execution controls: 23 semantic, one fixed-point source, one event-extraction source, four cross-artifact, six structural, two transport/invocation, two loaded-execution cache/code, 51 declared semantic/configuration-global mutations, and one CPython-3.11 normalization-removal source mutant on the affected route	All unenumerated verifier and Python faults
Certificate binds reviewed source inputs	Length-delimited source manifest and lockfile digest; stable regular-file reads; Cargo substitution rejection; registry source/checksum pins	Executable identity, native archives, ambient Cargo configuration, effective build

9.2 Trusted computing base

The producer-only value-enclosure trusted computing base includes:

- the intended SxPID event semantics and their transcription into the extractor;
- safe Rust source outside and inside the arithmetic wrapper;
- Rug, `gmp-mpfr-sys`, MPFR, and GMP;
- rustc, Cargo, build scripts, compiler wrappers, linker, native compiler, target ABI, and operating system;
- serialization and SHA-256 implementations;
- effective dependency features and native cache selection;
- input semantics and the consumer's acceptance procedure.

The independent containment route has a different trusted computing base:

- the reviewed Python verifier source, its event transcription, fixed lattice, exact rational algebra, and logarithm-series proof;

- the Python loader, bytecode compiler, process integrity, arbitrary-precision integer and `Fraction` semantics;
- strict JSON, SHA-256, filesystem reads, and the local producer-source manifest;
- the consumer's requirement that the verifier exit successfully and emit one complete canonical report.

The verifier records an observed-source digest and a digest of live module-owned code plus a deterministically typed, exact 51-name semantic/configuration-global inventory, and it rejects post-import source or execution drift. Qualification mutates each inventoried global and checks automatic discovery against the explicit list. These controls make named drift visible; they do not prove that the inventory is complete, Python's source-to-bytecode mapping, or process integrity. Before serializing code, verification schema v3 recursively primes CPython string-intern state so that a lazy cache transition cannot by itself change the digest. This is a runtime-specific normalization of a local drift detector, not a portable executable identity. The default marshal format, code-object representation, imported modules, interpreter, and process remain trusted. The CLI canonicalizes its own script name through `realpath`, binds the resulting target's exact initial bytes, and rechecks them during verification. Source-manifest members must be bounded regular non-symlink files read through one descriptor with identity checks before and after the read. The retained symlink-invocation control confirms that a symlinked script name binds the real target, while the qualification harness compiles the exact source bytes without a bytecode-cache lookup. These controls detect named local substitution and link cases; they are not an atomic filesystem snapshot, source-loader proof, or provenance attestation. The independent route removes the producer arithmetic stack from containment, not from producing the interval that is checked.

9.3 Why this is stronger than blind benchmarking

Blind holdout tests can detect specification and estimator failures that proofs miss, but they cannot prove an enclosure for an untested input. The current assurance case separates:

1. exact mathematical specification;
2. executable event and lattice reconstruction;
3. conditional numerical enclosure;
4. bounded independent-path agreement;
5. adversarial mutation sensitivity;
6. statistical estimator validation; and
7. downstream application qualification.

Only the first five layers are touched here, and each has a separate claim. Statistical and application evidence must not be inferred from software-correctness evidence.

10 Independent containment route and next formal-method frontier

10.1 Implemented integer/Fraction rational-log verifier

The standard-library verifier at `audit/tools/certified-sxpid/scripts/verify_certificate.py` treats the Rug certificate as untrusted. From canonical counts it reconstructs event

masses, the fixed two-source lattice, all cumulative and atom expressions, and three direct mutual-information identities. It requires the producer's 24 exact term lists to equal those independently reconstructed lists, but it does not use the producer lists to establish the values. Its target-restricted redundancy inclusion–exclusion uses complete-state uniqueness as stated in Equation (32). Its $ZM = I_4$ and zeta reconstruction checks are arithmetic self-consistency checks; the event transcription and semantic labels remain premises challenged separately by the direct row scans and mutations.

For every positive rational logarithm argument x, the verifier chooses the unique integer e and rational y such that

$$x = 2^e y, \qquad 1 \le y < 2,$$

and puts $z = (y-1)/(y+1)$. Hence $0 \le z < 1/3$. The identity and its positive remainder follow without an appeal to a numerical library:

$$\frac{d}{dz}\log\frac{1+z}{1-z} = \frac{2}{1-z^2} = 2\sum_{j=0}^{\infty} z^{2j}, \tag{63}$$

$$\log\frac{1+z}{1-z} = 2\sum_{j=0}^{m-1}\frac{z^{2j+1}}{2j+1} + R_m, \tag{64}$$

$$R_m = 2\sum_{r=0}^{\infty}\frac{z^{2m+2r+1}}{2m+2r+1} \tag{65}$$

$$\le \frac{2z^{2m+1}}{(2m+1)(1-z^2)} \le \frac{9z^{2m+1}}{4(2m+1)}. \tag{66}$$

The geometric series converges uniformly on the closed interval $[0, 1/3]$, so the displayed termwise integration is valid there. The last inequality uses $z \le 1/3$, so $(1-z^2)^{-1} \le 9/8$.

The cached $\log 2$ interval is constructed separately rather than by recursive range reduction. The same recurrence is applied at $y = 2$, for which

$$z = \frac{2-1}{2+1} = \frac{1}{3}, \qquad 2 = \frac{1+z}{1-z}.$$

Because Equation (66) includes the endpoint $z = 1/3$, this base calculation encloses $\log 2$.

The executable recurrence is also explicit. Fix a precision b and scale $S = 2^b$. For a real r, write

$$\lfloor r \rfloor_S := \frac{\lfloor Sr \rfloor}{S}, \qquad \lceil r \rceil_S := \frac{\lceil Sr \rceil}{S}.$$

The implementation stores integer units. It initializes

$$z_L = \lfloor Sz \rfloor, \qquad z_U = \lceil Sz \rceil, \tag{67}$$

$$w_L = \left\lfloor \frac{z_L^2}{S} \right\rfloor, \qquad w_U = \left\lceil \frac{z_U^2}{S} \right\rceil, \tag{68}$$

$$p_{0,L} = z_L, \qquad p_{0,U} = z_U, \tag{69}$$

$$L_0 = 0, \qquad U_0 = 0, \tag{70}$$

and, for $j = 0, \dots, m-1$, applies

$$L_{j+1} = L_j + \left\lfloor \frac{2p_{j,L}}{2j+1} \right\rfloor, \qquad U_{j+1} = U_j + \left\lceil \frac{2p_{j,U}}{2j+1} \right\rceil, \tag{71}$$

$$p_{j+1,L} = \left\lfloor \frac{p_{j,L} w_L}{S} \right\rfloor, \qquad p_{j+1,U} = \left\lceil \frac{p_{j,U} w_U}{S} \right\rceil. \tag{72}$$

All floors and ceilings in these formulas are exact integer divisions. Induction gives

$$\frac{p_{j,L}}{S} \le z^{2j+1} \le \frac{p_{j,U}}{S}$$

and

$$\frac{L_j}{S} \le 2\sum_{k=0}^{j-1} \frac{z^{2k+1}}{2k+1} \le \frac{U_j}{S}.$$

The lower induction step follows because multiplication is monotone on nonnegative operands and floor rounds down; the upper step is the same argument with ceiling. Addition and division by the positive odd integer preserve the endpoint order. Combining this invariant with Equation (66) returns

$$\left[\frac{L_m}{S}, \frac{U_m + \lceil 9p_{m,U}/(4(2m+1))\rceil}{S}\right]. \tag{73}$$

The program chooses

$$m = \max\left\{32, \left\lfloor \frac{b+32}{3} \right\rfloor + 1\right\}.$$

Range reduction adds $e \log 2$. If $e \ge 0$, the lower endpoint uses the lower enclosure of $\log 2$ and the upper endpoint uses the upper enclosure; if $e < 0$, those endpoints are swapped. For a rational coefficient $a = n/d$ with $d > 0$, write the integer-unit log interval as $[\ell/S, u/S]$. It is mapped to

$$\left[\frac{\lfloor n\ell/d \rfloor}{S}, \frac{\lceil nu/d \rceil}{S}\right] \quad \text{when } n \ge 0,$$

and to the same formula with ℓ and u exchanged when $n < 0$. Finite interval sums use exact integer addition. The verifier tries only

$$b \in (256, 384, 512, 768, 1024, 1536, 2048).$$

It succeeds only when every independently reconstructed interval is a subset of the corresponding producer interval. Overlap alone and precision-schedule exhaustion are rejections.

Theorem 10.1 (Conditional independent containment)**.** *Suppose the canonical input and local source bindings pass, the independently reconstructed expressions are correct, Python integer and* `Fraction` *operations have their specified semantics, and the exact-integer recurrence implements Equations (72)–(73). If the verifier returns* `verified`*, then every accepted producer interval contains the exact-real value of the corresponding independently reconstructed averaged SxPID2 coordinate.*

Proof. The count-to-expression route constructs each coordinate as a finite sum $\sum_j a_j \ln q_j$ with exact rational a_j and positive rational q_j. Range reduction and the displayed positive-tail bound enclose each $\ln q_j$. Multiplication by a rational coefficient uses the lower/upper order when $a_j \ge 0$ and swaps it when $a_j < 0$; exact integer floor and ceiling operations preserve outwardness. Finite outward addition encloses the complete expression. Acceptance requires this independently obtained interval to be contained in the producer interval. Transitivity of interval inclusion gives the result. □

The qualification harness does not merely compare against plausible decimal values. It computes an exact-`Fraction` partial sum and an exact rational upper bound on the tail

$$\frac{9z^{2m+1}}{4(2m+1)}$$

over a grid of reduced rational arguments and requires the fixed-point interval to contain that reference enclosure. A retained source mutant subtracts 70 fixed-point units from the $\ln 2$ upper endpoint. The exact-`Fraction` grid rejects that one-sided loss of outwardness. This retained negative control demonstrates sensitivity to this named source fault; it does not establish which other evidence routes would accept the mutant.

The verifier also binds reported arithmetic metadata to the local Cargo manifest and lockfile. It rejects Cargo `[patch]` and `[replace]` substitution, pins the registry source and checksum of both locked arithmetic packages, cross-checks the reported host against the embedded `rustc -vV` text, rejects duplicate keys, nonstandard numeric constants, overlong JSON integer tokens, and noncanonical decimal-string fields, enforces the producer's structural limits during reconstruction, and rejects source replacement after module import. These controls strengthen custody and fail-closed behavior. They do not formally verify Python, the source loader, the logarithm proof, or the count-to-event transcription.

10.2 Deductive and bounded refinement

A layered route can reduce different trust components:

1. extend Lean from finite categorical event bridges to the exact two-source cumulative and matrix specification;
2. use Kani next for bounded bit-precise rejection, indexing, overflow, and producer/verifier fixture obligations on the compiled Rust path;
3. prove a small pure integer/event kernel against the formal specification in Verus, Creusot, or another Rust-oriented deductive verifier;
4. have Rocq or another small-kernel checker validate independently generated logarithm inequalities; and
5. bind the compiled binary, effective dependency graph, native archives, compiler, and execution environment in a separate reproducible-build evidence envelope.

No layer should import the producer's conclusion as an assumption. The independent paths should meet only at a versioned mathematical statement and certificate schema.

10.3 Exact-zero completion and retained v1 failure

Revision v1's empty-map witness remains sound historical evidence but is not complete. Report v2 closes exact zero and strict sign for every coordinate admitted by the explicit product preflight: it clears the empirical denominator, forms the exact rational product in equation (46), and compares its numerator with its denominator. It does not factor integers. The independent verifier reconstructs the same decision using Python `Fraction`. Exhaustion over all 12,869 nonzero binary tables with total count at most eight finds no nonempty product-one expression below eight and exactly 16 at eight; all 16 are five-support, five-term net unique atoms. The minimized retained example above is checked through the live producer and verifier, and a resealed false exact-product sign is rejected.

This completion is intentionally bounded. A preflight-unavailable record is an abstention, not a sign result. The dyadic interval remains the magnitude enclosure in all cases and retains its own endpoint-derived decision even when an exact product separately resolves zero.

11 Release disposition

The source-only certifier is suitable as a research reference for canonical exact-count two-source categorical SxPID under its embedded conditional claim. It materially improves the ability to distinguish a computed sign from a certified sign and to reproduce all exact event and lattice inputs to the numerical calculation.

It is not a release authority for all of `pid-rs`. The following language remains unsupported:

- "formally verified pid-rs";

- “the `f64` implementation is certified”;
- “the returned interval is a statistical confidence interval”;
- “the continuous PID estimator is validated”; and
- “a downstream action is safe because PID passed.”

Targeted runtime tests now exercise reversed endpoint order, empty successive intersections, nonfinite authoritative endpoints, and both nontrivial exact event-nesting relations. One nonblocking operational hardening item remains explicit: define an atomic output transport or require consumers to enforce complete JSON, exit status, and payload-digest acceptance as one transaction.

12 Reproduction commands

From the repository root:

```
just certified-sxpid

CARGO_TARGET_DIR=target/certified-sxpid \
  cargo test --release --locked \
  --manifest-path audit/tools/certified-sxpid/Cargo.toml

CARGO_TARGET_DIR=target/certified-sxpid-msrv \
  cargo +1.89 test --locked \
  --manifest-path audit/tools/certified-sxpid/Cargo.toml

python3 scripts/generate-sxpid2-exhaustive-oracle.py
python3 audit/tools/certified-sxpid/scripts/check-independent-verifier.py
python3 -O \
  audit/tools/certified-sxpid/scripts/check-independent-verifier.py
python3 scripts/check-method-catalog.py
python3 scripts/check-software-identity.py
just formal-certified-sxpid2-assurance-pdf
```

The project CI route does not upload or cache a compiled certifier artifact. Rug, `gmp-mpfr-sys`, MPFR, and GMP bring LGPL-3.0-or-later obligations to a linked executable. Any binary distribution requires a separate license review and compliant source and relinking route.

A Registered producer static-policy mutation inventory

The 34 registered producer static-policy mutations are:

1. implicit MPFR conversion;
2. implicit logarithm rounding;
3. nearest rounding;
4. binary64 escape;
5. explicit unsafe-function surface;
6. raw `Float` outside the wrapper;
7. reordered raw-`Float` import;

8. aliased `Round` outside the wrapper;
9. unmanifested compile-time include;
10. unmanifested `include_str`;
11. unmanifested `include_bytes`;
12. aliased compile-time include;
13. unmanifested path module;
14. unmanifested conditional path module;
15. direct native-system dependency feature;
16. effective-feature report overclaim;
17. missing directed multiplication;
18. missing negative-coefficient endpoint swap;
19. non-strict positive-sign boundary;
20. non-strict negative-sign boundary;
21. shortened target-vector equality;
22. shortened source-vector equality;
23. removed keyed-row-to-target-union nesting;
24. removed target-union-to-source-union nesting;
25. redirected source-manifest path;
26. changed build-script route;
27. removed resource preflight;
28. restated rather than recomputed target-width result;
29. removed per-expression term limit;
30. removed incremental cumulative limit;
31. changed redundancy union to joint intersection;
32. changed Möbius coefficient;
33. added unmanifested Rust module; and
34. added unmanifested build-script helper.

B Evidence interpretation table

Evidence	Supports	Does not support
Exact runtime identities	Internal algebraic consistency	Correctness of every event definition
Directed-rounding argument	Conditional enclosure	Correctness of MPFR or compiled code
Exhaustive $N \leq 4$ corpus	Complete bounded diverse-path agreement	Larger alphabets, universal proof
Independent reconstruction and containment	Conditional containment of the exact averaged SxPID2 coordinates accepted by the verifier	Formal correctness of Python or the verifier proof
Exact-rational logarithm qualification	Enclosure on 975 argument/precision cases and sensitivity to the retained one-sided source mutant	Universal correctness of the logarithm implementation
Mutation suites	Sensitivity to named faults	Absence of unnamed faults
Locked source manifest	Source and lockfile drift detection	Binary or native archive identity
Narrow claim envelope	Machine-readable scope	Truth of unverified assumptions

References

[1] A. Makkeh, A. J. Gutknecht, and M. Wibral, Introducing a differentiable measure of pointwise shared information, *Physical Review E* 103, 032149 (2021), https://doi.org/10.1103/PhysRevE.103.032149.

[2] The MPFR Project, GNU MPFR manual, rounding modes and correctly rounded functions, https://www.mpfr.org/mpfr-current/mpfr.html.

[3] Rug project, Rug 1.30.0 documentation, arbitrary-precision floats and explicit rounding, https://docs.rs/rug/1.30.0/rug/.